HW1

4

We can simply create a z^* as a strict convex converge of x^*, y^*.

Here to create 5 optimal z^*, let $\lambda = [\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{5}}, \frac{1}{\sqrt{6}}, \frac{1}{\sqrt{7}}]$ and $z^* = \lambda x^*, (1-\lambda)y^*$ creates 5 non integer solutions. Using irrational λ guarentees a non-integer solution

5

a

Let variables $x_1, x_2, ..., x_m$ be binary LP variables to represent the n-arcs in our set A. A variable a_i is equal to 1 iff it is choosen as a part of our subset and it does not conflict with other variables in our subset and 0 otherwise. Since we are trying to find the largest subset, our objective function should be to maximize $\sum_i x_i$. To ensure no conflicting, we create n constraints, with each constraint corresponding to a number $1 - n$ (problem definition was unclear if we include 0 in our arcs but if that is the case simply add a new 0 constraint). For these constraints, x_i has a coefficient of 1 iff it's arc covers the integer corresponding to the constraint (which can easily be found by iterating through the integers each arc covers).

Thus our binary LP looks like this:

$\max \sum_i x_i$

s.t. $Ax \leq b$

$\forall i, x_i \in \{0, 1\}$

$b = 1_n$ which is a vector of length n of all 1's

And to formulate A:

$$a_{ij} = \begin{cases} 0 & \text{if the } j\text{th arc does not cover } i \\ 1 & \text{if the } j\text{th arc covers } i \end{cases}$$

b

$A = \{(1,1), (1,2), (1,3), (2,4), (2,5), (2,6), (2,7), (2,8), (3,9)\}$

```
from docplex.mp.model import Model

mdl = Model(name='binary_lp')

x1 = mdl.binary_var(name='x1')
x2 = mdl.binary_var(name='x2')
x3 = mdl.binary_var(name='x3')
x4 = mdl.binary_var(name='x4')
x5 = mdl.binary_var(name='x5')
x6 = mdl.binary_var(name='x6')
x7 = mdl.binary_var(name='x7')
x8 = mdl.binary_var(name='x8')
x9 = mdl.binary_var(name='x9')

mdl.add_constraint(x1 + x2 + x3 <= 1)
mdl.add_constraint(x2 + x3 + x4 + x5 + x6 + x7 + x8 <= 1)
mdl.add_constraint(x3 + x4 + x5 + x6 + x7 + x8 + x9 <= 1)
mdl.add_constraint(x4 + x5 + x6 + x7 + x8 + x9 <= 1)
mdl.add_constraint(x5 + x6 + x7 + x8 + x9 <= 1)
mdl.add_constraint(x6 + x7 + x8 + x9 <= 1)
mdl.add_constraint(x7 + x8 + x9 <= 1)
mdl.add_constraint(x8 + x9 <= 1)
mdl.add_constraint(x9 <= 1)

mdl.maximize(x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9)

solution = mdl.solve()

print(solution)
```

This results in an optimal solution of $x_1 = 1, x_9 = 1$ with an objective of 2

c

The linear relaxation for this LP looks like this:

$\max \sum_i x_i$

s.t. $Ax \leq b$

$\forall i, 0 \leq x_i \leq 1$

$b = 1_n$ which is a vector of length n of all 1's

And to formulate A:

$$a_{ij} = \begin{cases} 0 & \text{if the } j\text{th arc does not cover } i \\ 1 & \text{if the } j\text{th arc covers } i \end{cases}$$

Thus for our case the linear relaxation is:

$\max \sum_i x_i$

s.t.$x_1 + x_2 + x_3 \leq 1$

$$x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 \leq 1$$
$$x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 \leq 1$$
$$x_4 + x_5 + x_6 + x_7 + x_8 + x_9 \leq 1$$
$$x_5 + x_6 + x_7 + x_8 + x_9 \leq 1$$
$$x_6 + x_7 + x_8 + x_9 \leq 1$$
$$x_7 + x_8 + x_9 \leq 1$$
$$x_8 + x_9 \leq 1$$
$$x_9 \leq 1$$
$$\forall i 0 \leq x_i \leq 1$$

d

$A = \{(1,2),(2,3),(3,4),(4,6),(5,7),(6,8),(7,9),(8,1),(9,3)\}$

```
from docplex.mp.model import Model

mdl = Model(name='binary_lp')

x1 = mdl.binary_var(name='x1')
x2 = mdl.binary_var(name='x2')
x3 = mdl.binary_var(name='x3')
x4 = mdl.binary_var(name='x4')
x5 = mdl.binary_var(name='x5')
x6 = mdl.binary_var(name='x6')
x7 = mdl.binary_var(name='x7')
x8 = mdl.binary_var(name='x8')
```

```
x9 = mdl.binary_var(name='x9')

mdl.add_constraint(x1 + x8 + x9 <= 1)
mdl.add_constraint(x1 + x2 + x9 <= 1)
mdl.add_constraint(x2 + x3 + x9 <= 1)
mdl.add_constraint(x3 + x4 <= 1)
mdl.add_constraint(x4 + x5 <= 1)
mdl.add_constraint(x4 + x5 + x6 <= 1)
mdl.add_constraint(x5 + x6 + x7 <= 1)
mdl.add_constraint(x6 + x7 + x8 <= 1)
mdl.add_constraint(x7 + x8 + x9 <= 1)

mdl.maximize(x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9)

solution = mdl.solve()

print(solution)
```

This results in an optimal solution of $x_1 = 1, x_3 = 1, x_5 = 1$ and an objective value of 3

e

The linear relaxation of this problem is as follows:

$\max \sum_i x_i$

s.t.$x_1 + x_8 + x_9 \leq 1$

mdl.add_constraint(x1 + x2 + x9 <= 1) mdl.add_constraint(x2 + x3 + x9 <= 1) mdl.add_constraint(x3 + x4 <= 1) mdl.add_constraint(x4 + x5 <= 1) mdl.add_constraint(x4 + x5 + x6 <= 1) mdl.add_constraint(x5 + x6 + x7 <= 1) mdl.add_constraint(x6 + x7 + x8 <= 1) mdl.add_constraint(x7 + x8 + x9 <= 1)

mdl.maximize(x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9)

6

a

Suppose there exists an optimal solution S with arc a_r appearing in two districts d_1, d_2. Create an S' by removing a_r from d_2, this is still a valid solution because all arcs are being represented and removing an arc from a district cannot introduce any sort of conflicting arcs. We also

know that S and S' are both optimal solutions because the number of districts did not change from S to S' and we defined S as the optimal solution. We can repeat this process recursively until we end with an S^* with each arc appearing a single time and similarly to the S' idea, S^* is also an optimal solution because no new districts were created.

b

Omit as solved in HW2 **5.a**

c

$A = \{(1,1),(1,2),(1,3),(2,4),(2,5),(2,6),(2,7),(2,8),(3,9)\}$

d

e

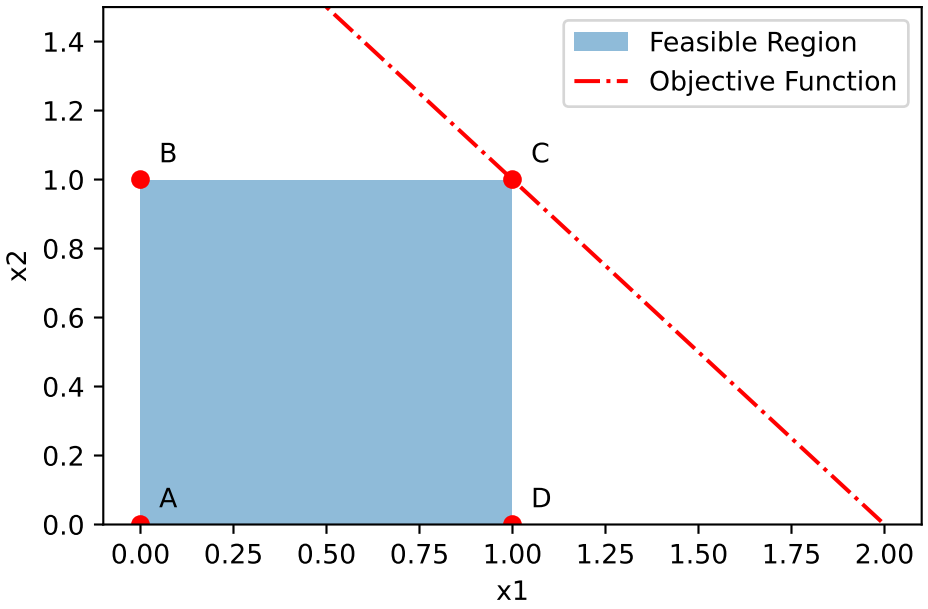
$A = \{(1,2),(2,3),(3,4),(4,6),(5,7),(6,8),(7,9),(8,1),(9,3)\}$

f

HW2

1

a



b

Remember the standard form is (from HW1 with slight changes for consistency):

$\min -x_1 - x_2$

s.t. $2(x_1) + (x_2) + s_3 = 3$

$x_1 + s_1 = 1$

$x_2 + s_2 = 1$

$x_1, s_1, x_2, s_2, s_3 \geq 0$

So we can write A as:

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix}, C = \begin{pmatrix} -1 \\ -1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Thus we can create our B and L for the origin as:

$$B = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad L = \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Thus we have:

$$\pi = (B^{-1})^T C_B = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}^T \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

And it trivially follows that:

$$C^\pi = C - A^T \pi = \begin{pmatrix} -1 \\ -1 \\ 0 \\ 0 \\ 0 \end{pmatrix} - A^T \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

c

$$\text{A BFS for the optimal point is } \bar{x} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Thus we can create our B and L for the optimal point $\bar{x}$ as:

$$B = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad L = \begin{pmatrix} 0 & 1 \\ 0 & 0 \\ 1 & 0 \end{pmatrix}$$

Thus we have:

$$\pi = (B^{-1})^T C_B = \begin{pmatrix} 0.5 & 0 & -0.5 \\ 0 & 0 & 1 \\ -0.5 & 1 & 0.5 \end{pmatrix}^T \begin{pmatrix} -1 \\ -1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.5 & 0 & -0.5 \\ 0 & 0 & 1 \\ -0.5 & 1 & 0.5 \end{pmatrix} \begin{pmatrix} -1 \\ -1 \\ 0 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0 \\ -0.5 \end{pmatrix}$$

And it trivially follows that:

$$C^\pi = C - A^T\pi = \begin{pmatrix} -1 \\ -1 \\ 0 \\ 0 \\ 0 \end{pmatrix} - A^T \begin{pmatrix} -0.5 \\ 0 \\ -0.5 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 0 \\ 0 \\ 0 \end{pmatrix} - \begin{pmatrix} -1 \\ -1 \\ 0 \\ -0.5 \\ -0.5 \end{pmatrix} = \begin{pmatrix} -2 \\ -2 \\ 0 \\ -0.5 \\ -0.5 \end{pmatrix}$$

d

We can find the basic solutions through the following formula then check if the solution is a feasible one:

$$b = \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix}, x_B = \begin{pmatrix} B^{-1}b \\ 0 \end{pmatrix}$$

Naively there are 5 choose 3 different basis but there are a few basis that are invalid as they dont span R^3. Namely these were the basis that span R^3

$$B_1 = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, B_2 = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{pmatrix}, B_3 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, B_4 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

$$B_5 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} B_6 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}, B_7 = \begin{pmatrix} 2 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, B_8 = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Thus we can find all of the B^{-1}:

$$B_1^{-1} = \begin{pmatrix} 0.5 & 0 & -0.5 \\ 0 & 0 & 1 \\ -0.50 & 1 & 0.5 \end{pmatrix}, B_2^{-1} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -2 & 0 \\ -1 & 2 & 1 \end{pmatrix}, B_3^{-1} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & -2 & -1 \end{pmatrix}, B_4^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}$$

$$B_5^{-1} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} B_6^{-1} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{pmatrix}, B_7^{-1} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & -2 & 0 \end{pmatrix}, B_8^{-1} = \begin{pmatrix} 0.5 & 0 & 0 \\ -0.5 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Let x^i denote the basic solution for basis B_i

$$x^1 = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, x^2 = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, x^3 = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, x^4 = \begin{pmatrix} 0 \\ 3 \\ 1 \\ -2 \\ 0 \end{pmatrix}, x^5 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 3 \end{pmatrix}, x^6 = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 2 \end{pmatrix}, x^7 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}, x^8 = \begin{pmatrix} 1.5 \\ 0 \\ -0.5 \\ 1 \\ 0 \end{pmatrix}$$

Of these it is clear that x^1, x^4, x^8 are not feasible solutions.

e

Omit, moved to HW3

f

Omit, moved to HW3

2

a

We can reuse the same LP from the previous question but change the cost function to be the minimum of $x_1 + x_2$ thus in standard form we have:

$\min x_1 + x_2$

s.t. $2(x_1) + (x_2) + s_3 = 3$

$x_1 + s_1 = 1$

$x_2 + s_2 = 1$

$x_1, s_1, x_2, s_2, s_3 \geq 0$

So our extreme point is

$$w = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Wth $B_1 = x_1, x_2, s_1$ and $B_2 = x_1, x_2, s_2$

b

Our program has 5 variables so $q = 5$

i

The only nonzero variables are x_1, x_2 so

$$A_1 = A_2 = \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}$$

This is 2 columns which is less than $q = 5$

ii

The columns are linearly independent because the first column does not have an entry in the 3rd row so it cannot be multiplied by a scalar in order to reach the second column.

iii

We can use any of the basis B_1, B_2, B_3 from the previous question which adds a single new column to A_1 and correspond to the same extreme point w. Explicitly we can use B_1 by adding the third column of A to A_1 to get

$$A_1 = A_2 = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

3

a

```
from docplex.mp.model import Model

mdl = Model(name='basic_lp')

x1 = mdl.continuous_var(name='x1', lb=0)
x2 = mdl.continuous_var(name='x2', lb=0)
x3 = mdl.continuous_var(name='x3', lb=0)
x4 = mdl.continuous_var(name='x4', lb=0)
gamma = mdl.continuous_var(name='gamma')

mdl.minimize(gamma)

mdl.add_constraint(3*x1 + 4*x2 + 2*x3 + 2*x4 <= 2*gamma)
mdl.add_constraint(3*x1 + 4*x2 + 5*x3 + 3*x4 <= 3*gamma)
mdl.add_constraint(3*x1 + 4*x2 + 5*x3 + 6*x4 <= 3*gamma)
mdl.add_constraint(x1 + x2 + x3 + x4 == 1)

solution = mdl.solve()
print(solution)
```

This resulted in a $\gamma = 1.286$ as the objective and $x_1 = 0.571, x_3 = 0.429$

b

```
best_gamma = mdl.solution.objective_value

mdl.add_constraint(gamma<=best_gamma+0.001)

mdl.maximize(x4)

solution = mdl.solve()
print(mdl)
print(solution)
```

Unfortunately due to rounding errors the python CPLEX API did not work for an exact $\gamma = 1.286$ and instead returned the same solution of $x_1 = 0.571, x_3 = 0.429, x_4 = 0$. To cut

some slack I added a 0.001 error to our constraint and found that for solution $\gamma = 0.1287, x_1 = 0.573, x_3 = 0.42, x_4 = 0.007$. However due to this small increase it appears more likely that there is not a nonzero x_4 that results in the optimal solution of $\gamma = 0.1286$.

c

Omit, moved to HW3

d

Omit, moved to HW3

e

4

a

$\min s_1 + s_2$

s.t. $2x_1 + x_2 + s_3 = 3$

$x_1 + s_1 = 1$

$x_2 + s_2 = 1$

$x_1, x_2, s_1, s_2, s_3 \geq 0$

b

We can convert this into standard form by adding vector s such taht $|x| = |s|$ and we add new constraints: $x + s = u$ and of course $x, s \geq 0$. To create our new cost vector $\tilde{c}$ where $|\tilde{c}| = |x| + |s|$ and we create it using the following piecewise function by flipping the sign of any negative costs and assigning that cost to the corresponding slack variable:

$$\tilde{c}_i = \begin{cases} 0 & \text{if } i \leq |x| \wedge c_i < 0 \\ c_i & \text{if } i \leq |x| \wedge c_i \geq 0 \\ 0 & \text{if } i > |x| \wedge c_{i-|x|} \geq 0 \\ -c_{i-|x|} & \text{if } i > |x| \wedge c_{i-|x|} < 0 \end{cases}$$

Thus our cost vector has only positive components and our new LP is as follows and is an equivalent minimization problem:

$$\begin{aligned} &\min \tilde{c}^T \begin{pmatrix} x \\ s \end{pmatrix} \\ &\text{s.t. } Ax = b \\ &x + s = u \\ &x, s \geq 0 \end{aligned}$$

5

a

Omit, moved to HW3

b

c

d